# Домашнее задание №4

## Требования и архитектурное мышление

**Система: Notification Platform для онлайн-банка**

---

# 1. Функциональные требования

### 1. Критичные уведомления

Как пользователь, я хочу получать уведомления о транзакциях (списания, переводы) практически мгновенно, чтобы контролировать состояние своего счета.

### 2. Управление настройками уведомлений

Как пользователь, я хочу настраивать, какие типы уведомлений я получаю, чтобы отключать ненужные сообщения.

### 3. Доставка через разные каналы

Как пользователь, я хочу получать уведомления через разные каналы (push, SMS, email), чтобы иметь возможность выбрать наиболее удобный способ получения информации.

### 4. Массовые уведомления

Как банк, я хочу отправлять уведомления большому количеству пользователей (до 1 млн), чтобы информировать их об акциях и предложениях.

### 5. Гарантия доставки критичных сообщений

Как банк, я хочу гарантировать доставку критичных уведомлений, чтобы обеспечить надежность финансовых операций и снизить риски.

### 6. Избежание дублирования уведомлений

Как банк, я хочу, чтобы система не отправляла дублирующиеся уведомления, чтобы улучшить пользовательский опыт и снизить количество жалоб.

### 7. Централизованная отправка уведомлений

Как банк, я хочу, чтобы все уведомления отправлялись через единую платформу, чтобы обеспечить контроль и единый подход к коммуникации с пользователями.

---

# 2. Нефункциональные требования

## 1. Производительность

Система должна обеспечивать доставку:

- транзакционных уведомлений — не более **1 секунды (P95)**
- сервисных уведомлений — не более **60 секунд (P95)**

## 2. Масштабируемость

Система должна поддерживать отправку массовых уведомлений до **1 000 000 пользователей** в рамках одной кампании без деградации производительности (время обработки — не более **30 минут**).

## 3. Надежность доставки

Система должна обеспечивать доставку не менее **99.9%** всех уведомлений, включая повторные попытки отправки при сбоях.

## 4. Доступность

Система должна иметь доступность не ниже **99.95%** в месяц для обработки и отправки уведомлений.

## 5. Отказоустойчивость

При недоступности внешнего провайдера (SMS/email) система должна:

- выполнять повторные попытки отправки (не менее **3 попыток**)
- обеспечивать переключение на альтернативный канал в течение **5 секунд**

## 6. Обработка ошибок

Система должна фиксировать **100% ошибок доставки** и обеспечивать возможность повторной обработки сообщений из очереди ошибок.

## 7. Пропускная способность

Система должна обрабатывать не менее 10 000 уведомлений в секунду при пиковых нагрузках.

---

# 3. Архитектурно значимые требования (ASR)

## ASR 1: Низкая задержка доставки транзакционных уведомлений

**Связанные требования:**

- доставка транзакционных уведомлений ≤ 1 секунды (P95)
- высокая критичность уведомлений (пользователь контролирует финансы)

**Почему влияет на архитектуру:**
Требует асинхронной обработки и минимизации задержек, что приводит к использованию очередей сообщений и выделению отдельного приоритетного потока обработки для критичных уведомлений.

## ASR 2: Высокая масштабируемость для массовых рассылок

**Связанные требования:**

- обработка до 1 000 000 пользователей
- пропускная способность ≥ 10 000 уведомлений в секунду

**Почему влияет на архитектуру:**
Требует горизонтального масштабирования компонентов, использования очередей и распределенной обработки, а также разделения нагрузки на батчи.

## ASR 3: Гарантированная доставка уведомлений

**Связанные требования:**

- уровень доставки ≥ 99.9%
- повторные попытки отправки (не менее 3 попыток)
- обработка ошибок

**Почему влияет на архитектуру:**
Требует внедрения механизмов повторной отправки, хранения состояния сообщений и использования очередей с гарантией доставки, что усложняет архитектуру и добавляет дополнительные компоненты.

## ASR 4: Отказоустойчивость при сбоях провайдеров

**Связанные требования:**

- переключение на резервный канал ≤ 5 секунд
- работа при нестабильных внешних сервисах

**Почему влияет на архитектуру:**
Требует интеграции с несколькими провайдерами и реализации логики переключения, что увеличивает сложность системы и влияет на маршрутизацию сообщений.

---

# 4. Ключевые архитектурные вопросы

## Вопрос 1: Как обеспечить низкую задержку доставки транзакционных уведомлений?

**Связанные требования / ASR:**

- доставка ≤ 1 секунды (P95)
- ASR: низкая задержка критичных уведомлений

**Почему это важно:**
Задержка напрямую влияет на пользовательский опыт и доверие к банку. Неправильный выбор архитектуры (например, синхронная обработка или перегруженные компоненты) может привести к задержкам и нарушению SLA.

## Вопрос 2: Как обеспечить масштабируемость системы при массовых рассылках?

**Связанные требования / ASR:**

- обработка до 1 000 000 пользователей
- пропускная способность ≥ 10 000 уведомлений в секунду
- ASR: высокая масштабируемость

**Почему это важно:**
Система должна выдерживать пиковые нагрузки без деградации. Неправильные архитектурные решения могут привести к перегрузке системы и сбоям при массовых кампаниях.

## Вопрос 3: Как гарантировать доставку уведомлений при сбоях внешних провайдеров?

**Связанные требования / ASR:**

- уровень доставки ≥ 99.9%
- не менее 3 повторных попыток отправки
- ASR: гарантированная доставка

**Почему это важно:**
Внешние провайдеры могут работать нестабильно, и без механизмов повторной отправки и переключения между каналами часть уведомлений может быть потеряна, что критично для банковской системы.

## Вопрос 4: Как избежать дублирования уведомлений?

**Связанные требования / ASR:**

- отсутствие дублирующихся уведомлений
- централизованная система отправки

**Почему это важно:**
Дубли ухудшают пользовательский опыт и увеличивают нагрузку на систему и провайдеров.

---

# 5. Архитектурные последствия ASR

## ASR 1: Низкая задержка доставки транзакционных уведомлений

**Последствия:**

- использование асинхронной обработки (очереди сообщений)
- выделение отдельного приоритетного потока для транзакционных уведомлений
- минимизация количества промежуточных сервисов
- возможность обработки в реальном времени

## ASR 2: Высокая масштабируемость для массовых рассылок

**Последствия:**

- использование распределенной системы (несколько воркеров)
- горизонтальное масштабирование компонентов
- обработка сообщений батчами
- применение ограничения скорости обработки запросов для защиты от перегрузки

## ASR 3: Гарантированная доставка уведомлений

**Последствия:**

- внедрение механизма повторной отправки уведомлений
- использование очередей с гарантией доставки
- хранение состояния сообщений
- использование очереди сообщений с ошибками

## ASR 4: Отказоустойчивость при сбоях провайдеров

**Последствия:**

- интеграция с несколькими внешними провайдерами
- реализация механизма переключения между провайдерами
- переключение на альтернативные каналы доставки
- мониторинг состояния внешних сервисов

---

# 6. Архитектурные решения, которые НЕ подходят

## Решение 1: Синхронная отправка уведомлений напрямую из бизнес-сервисов

**Какое решение:**
Каждый сервис (платежи, кредиты и т.д.) напрямую отправляет уведомления через SMS/email API без использования очередей.

**Какой ASR нарушается:**

- ASR: низкая задержка при высокой нагрузке
- ASR: масштабируемость
- ASR: гарантированная доставка

**Почему:**
При высокой нагрузке сервисы будут блокироваться на внешних вызовах, что увеличит задержки и приведет к сбоям. Также отсутствуют механизмы повторной отправки и централизованного контроля, что снижает надежность доставки.

## Решение 2: Использование одного внешнего провайдера без резервирования

**Какое решение:**
Система работает только с одним SMS/email провайдером без возможности переключения.

**Какой ASR нарушается:**

- ASR: отказоустойчивость
- ASR: гарантированная доставка

**Почему:**
При сбое провайдера доставка уведомлений полностью останавливается. Это критично для банковской системы, где необходимо обеспечивать надежную доставку сообщений.

## Решение 3: Отправка всех уведомлений в одном потоке без приоритизации

**Какое решение:**
Все типы уведомлений (транзакционные, сервисные, маркетинговые) обрабатываются одинаково без разделения по приоритетам.

**Какой ASR нарушается:**

- ASR: низкая задержка для критичных уведомлений

**Почему:**
При массовых рассылках маркетинговые уведомления могут перегружать систему и задерживать доставку критичных сообщений.

---

# 7. Неопределённости и архитектурные риски

## Неопределённость 1: Реальная пропускная способность внешних провайдеров

**Что неизвестно:**
Какую фактическую нагрузку (запросов в секунду) могут выдерживать SMS/email провайдеры без деградации или блокировок.

**Как проверить:**

- провести нагрузочное тестирование
- запросить SLA и ограничения у провайдеров

- протестировать поведение при пиковых нагрузках

## Неопределённость 2: Поведение системы при пиковых нагрузках

**Что неизвестно:**
Сможет ли система стабильно обрабатывать до 1 млн уведомлений без задержек и сбоев.

**Как проверить:**

- провести стресс-тестирование
- использование тестовой среды с имитацией реальной нагрузки
- измерить производительность и пропускную способность

## Неопределённость 3: Предпочтения пользователей по каналам уведомлений

**Что неизвестно:**
Какие каналы (push, SMS, email) пользователи реально предпочитают и насколько активно отключают уведомления.

**Как проверить:**

- провести A/B тестирование
- анализировать пользовательскую статистику
- собрать аналитику по открытию и доставке уведомлений

---

# 8. RFC: Гарантированная доставка критичных уведомлений с кросс-канальным failover

| Метаданные | Значение |
|---|---|
| **Статус** | DRAFT |
| **Автор(ы)** | Сополев Владислав |
| **Ответственный** | Сополев Владислав |
| **Бизнес-заказчик** | N/A |
| **Ревьюеры** | N/A |
| **Дата создания** | 2026-04-06 |
| **Дата обновления** | 2026-04-06 |

# Контекст

**Проблема:**
Транзакционные уведомления (подтверждения переводов, списания средств) являются критичными для пользователя. В текущей системе уведомления рассылались разными сервисами напрямую, что приводило к:

- Задержкам доставки
- Потере сообщений при сбоях внешних провайдеров (SMS/email/push)
- Дублированию уведомлений
- Невозможности контролировать SLA

**Цель:**
Создать подсистему гарантированной доставки критичных уведомлений, которая:

- Минимизирует задержку доставки
- Обеспечивает failover между каналами (SMS → email → push)
- Учитывает пользовательские настройки предпочтений
- Предотвращает дублирование сообщений
- Поддерживает наблюдаемость и контроль доставки

**Ключевые вопросы:**

- Как гарантировать доставку даже при отказе внешних провайдеров?
- Как минимизировать задержку критичных уведомлений?
- Как не перегрузить систему при массовых рассылках?

# Требования

## Функциональные требования

| № | Приоритет | Обозначение | Требование |
|---|---|---|---|
| 1 | MUST HAVE | FR1 | Система должна доставлять транзакционные уведомления пользователю через любой доступный канал (push/SMS/email). |
| 2 | MUST HAVE | FR2 | Система должна автоматически переключаться на резервный канал при недоступности основного. |
| 3 | MUST HAVE | FR3 | Система должна предотвращать дублирование уведомлений при failover. |
| 4 | SHOULD HAVE | FR4 | Система должна учитывать пользовательские настройки предпочтительного канала и отключения уведомлений. |
| 5 | MUST HAVE | FR5 | Система должна обеспечивать наблюдаемость доставки: логирование, метрики, трейсинг. |

## Нефункциональные требования

| № | Приоритет | Обозначение | Требование |
|---|---|---|---|
| 1 | MUST HAVE | NFR1 | SLA для транзакционных уведомлений ≤ 1 секунда (P95). |

| № | Приоритет | Обозначение | Требование |
|---|---|---|---|
| 2 | MUST HAVE | NFR2 | Доставка ≥ 99.9% уведомлений, включая повторные попытки (retry ≥ 3). |
| 3 | MUST HAVE | NFR3 | Система должна выдерживать пропускную способность ≥ 10 000 уведомлений/сек. |
| 4 | MUST HAVE | NFR4 | Failover между каналами должен происходить ≤ 5 секунд. |
| 5 | SHOULD HAVE | NFR5 | Система должна масштабироваться до 1 млн пользователей в массовой кампании без деградации производительности. |

# Архитектурно значимые требования (ASR)

| ASR | Связанные требования | Влияние на архитектуру |
|---|---|---|
| ASR1: Низкая задержка доставки критичных уведомлений | FR1, NFR1 | Требует приоритизации транзакционных уведомлений, асинхронной обработки, выделенных потоков и очередей с минимальными промежуточными сервисами. |
| ASR2: Гарантированная доставка уведомлений | FR2, FR3, NFR2 | Требует хранения состояния сообщений, retry-механизмов, очередей с гарантией доставки, предотвращения дублирования и очереди сообщений с ошибками. |
| ASR3: Отказоустойчивость при сбоях провайдеров | FR2, FR3, NFR4 | Требует интеграции с несколькими каналами, failover логики, мониторинга состояния провайдеров и маршрутизации сообщений. |
| ASR4: Масштабируемость системы | NFR3, NFR5 | Требует горизонтального масштабирования компонентов, батчевой обработки, распределенных очередей и ограничения скорости обработки запросов. |

# Варианты решения

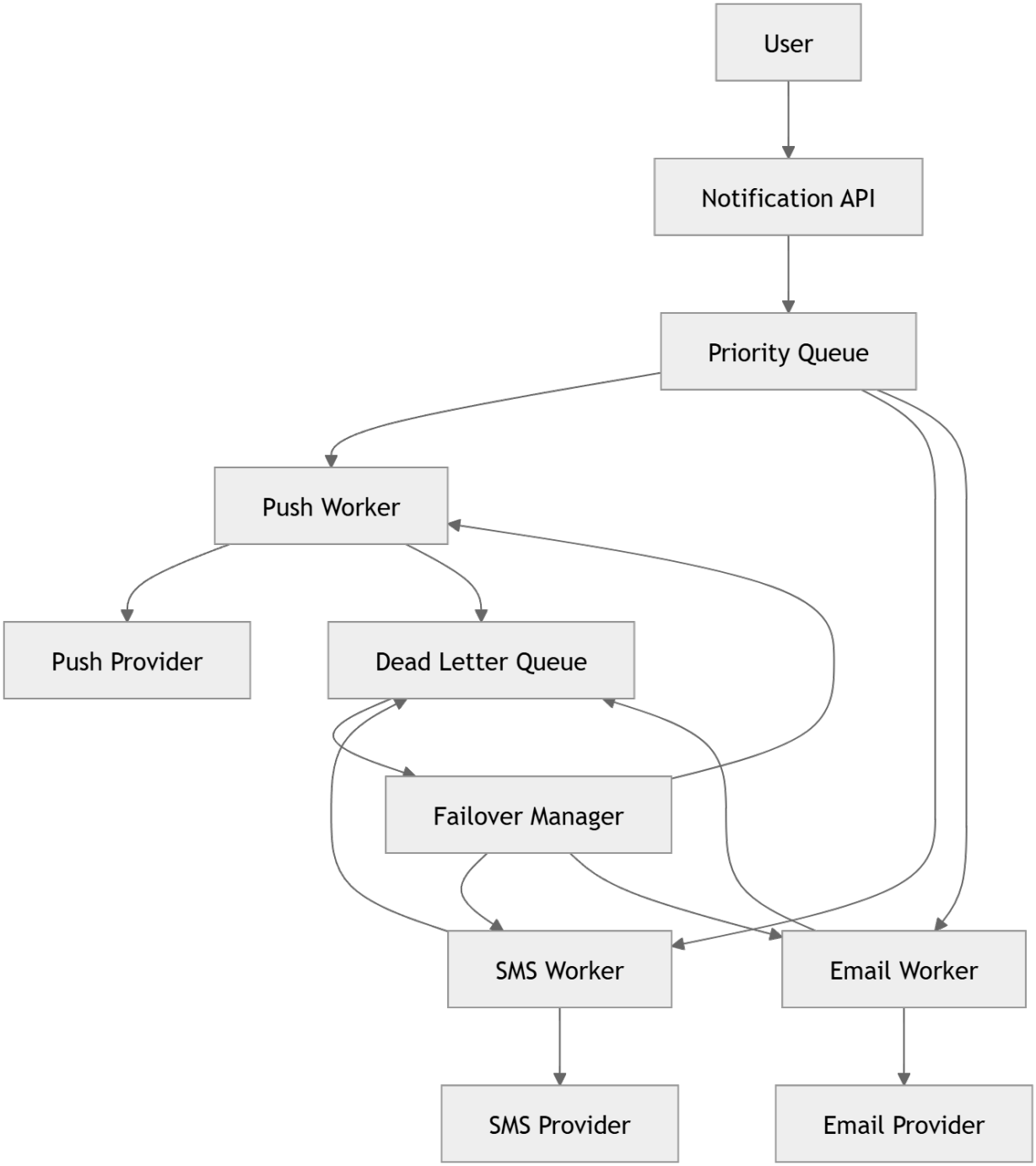
## Вариант 1: Центральная очередь с приоритетами и failover

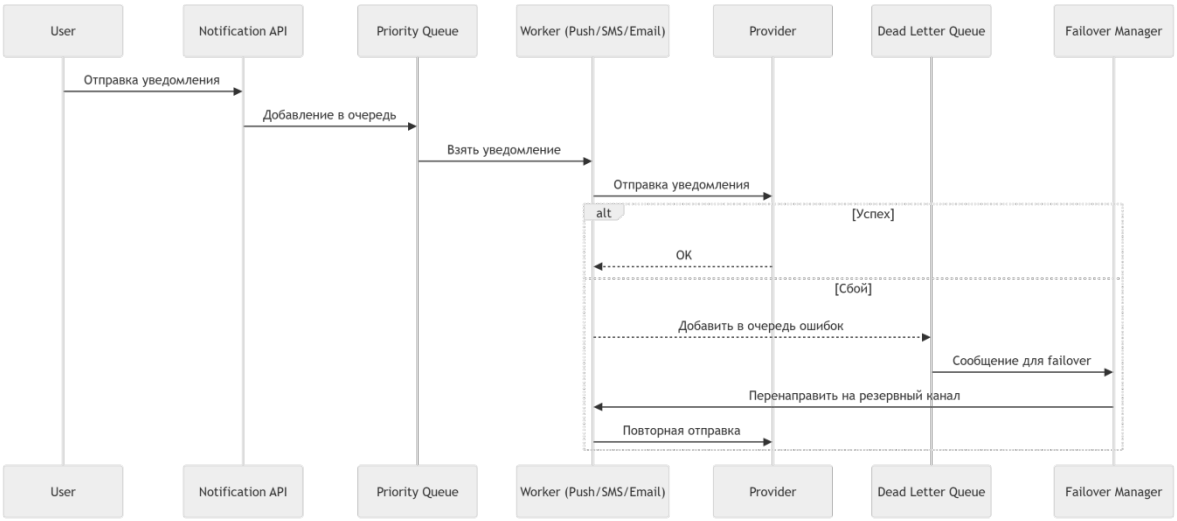
**Описание:** Все критичные уведомления попадают в централизованную очередь сообщений (например, RabbitMQ/Kafka). Каждое сообщение хранит состояние доставки по каналам и приоритет. Отдельные воркеры обрабатывают уведомления для push, SMS и email. Если основной канал недоступен, система автоматически перенаправляет сообщение на резервный канал.

**Архитектура**

- Централизованная очередь сообщений с поддержкой приоритетов.
- Воркеры для каждого канала (push/SMS/email) с retry и Dead Letter Queue.
- Фейловер-менеджер контролирует доступность провайдеров и перенаправляет уведомления.

- Логирование и метрики (Prometheus, Grafana) для наблюдаемости.






### Этапы реализации

| Этап | Описание | Планируемый срок | Ресурсы | Риски |
|---|---|---|---|---|
| 1 | Развернуть очередь сообщений с поддержкой приоритетов | 2 недели | DevOps, очередь (Kafka/RabbitMQ) | Возможные задержки при высокой нагрузке |
| 2 | Реализовать воркеры для каждого канала с retry | 3 недели | Backend команда | Ошибки в логике failover, дублирование |
| 3 | Интеграция с внешними провайдерами и failover | 2 недели | Backend, QA | Провайдеры нестабильны |
| 4 | Настройка мониторинга и метрик | 1 неделя | DevOps | Неполная наблюдаемость |

### Преимущества

- Надежная доставка, даже при сбоях внешних провайдеров
- Приоритизация критичных уведомлений
- Возможность наблюдать и логировать каждый этап доставки

### Недостатки

- Более высокая сложность архитектуры
- Требуется управление несколькими очередями и воркерами
- Возможны задержки при массовых рассылках при пиковых нагрузках

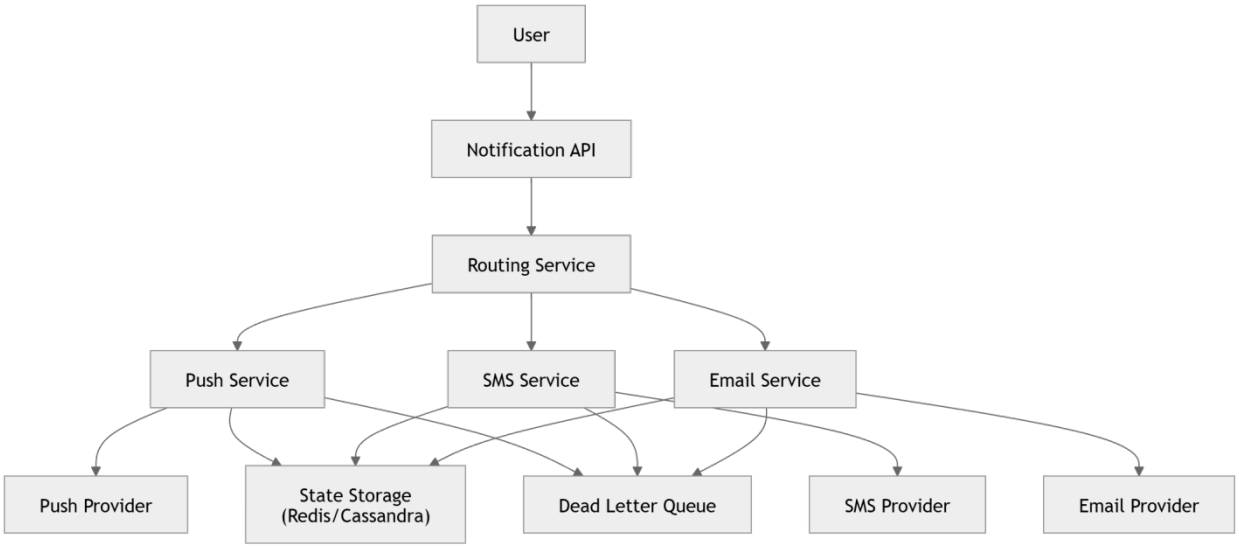
## Вариант 2: Микросервисная маршрутизация с резервными каналами

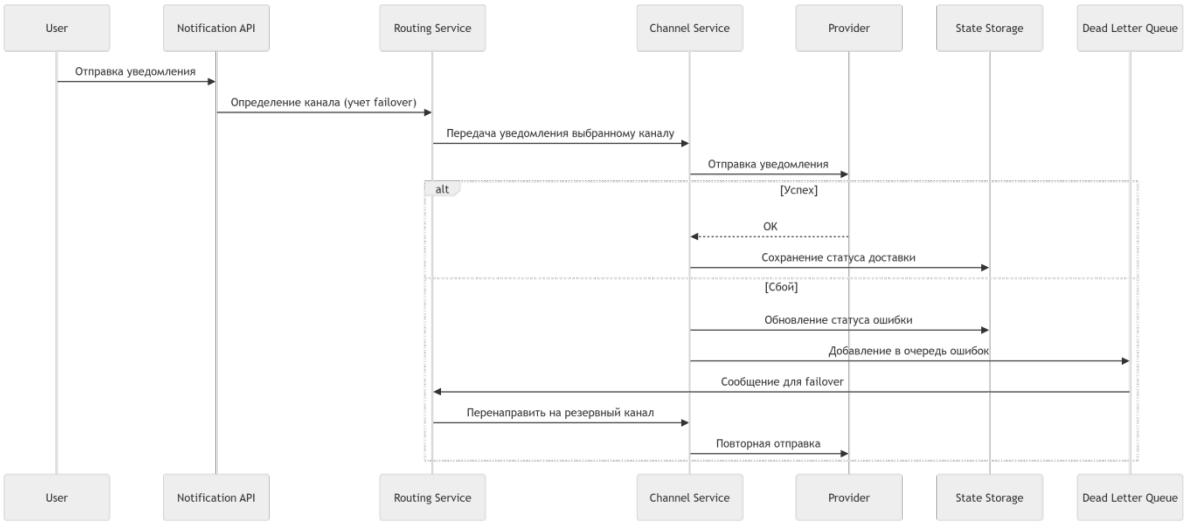
**Описание:** Система делится на микросервисы: API прием уведомлений, сервис маршрутизации и сервисы каналов (push/SMS/email). Сервис маршрутизации решает, через какой канал отправить уведомление с учетом приоритета, пользовательских

настроек и доступности провайдера. Каждое уведомление хранится в распределенном хранилище состояния (например, Redis или Cassandra), чтобы обеспечить retry и failover.

**Архитектура**

- Notification API принимает события.
- Routing Service определяет канал для отправки, учитывает failover.
- Channel Service отправляет уведомление, логирует результат.
- Хранилище состояния обеспечивает retry и предотвращение дублирования.




## Этапы реализации

| Этап | Описание | Планируемый срок | Ресурсы | Риски |
|---|---|---|---|---|
| 1 | Разработать API и сервис | 2 недели | Backend | Ошибки маршрутизации, |

| Этап | Описание | Планируемый срок | Ресурсы | Риски |
|---|---|---|---|---|
| | маршрутизации | | | сложная логика |
| 2 | Реализовать сервисы каналов с retry и failover | 3 недели | Backend | Дублирование сообщений, сбои при массовой нагрузке |
| 3 | Настроить хранилище состояния и Dead Letter Queue | 2 недели | DevOps, Backend | Потеря состояния при сбоях |
| 4 | Мониторинг, метрики, трассировка | 1 неделя | DevOps | Неполное покрытие ошибок |

### Преимущества

- Четкая сегрегация ответственности между микросервисами
- Гибкая маршрутизация и failover
- Масштабируемость каждого сервиса независимо

### Недостатки

- Более высокая сложность интеграции
- Зависимость от распределенного хранилища состояния
- Возможны проблемы согласованности при массовых рассылках

## Сравнительный анализ

| Критерий | Вариант 1: Очередь с приоритетами | Вариант 2: Микросервисы с маршрутизацией |
|---|---|---|
| Простота реализации | Средняя | Сложная |
| Масштабируемость | Горизонтальная через воркеры | Горизонтальная через микросервисы |
| Надежность доставки | Высокая | Высокая |
| Управление приоритетами | Очередь с приоритетами | Routing Service |
| Failover | Через воркеры и менеджер failover | Через сервис маршрутизации и хранилище состояния |
| Наблюдаемость | Логи + метрики | Логи + метрики + распределенный трейсинг |
| Риск дублирования | Минимальный при | Минимальный при корректной |

| Критерий | Вариант 1: Очередь с приоритетами | Вариант 2: Микросервисы с маршрутизацией |
|---|---|---|
| | корректном DLQ | синхронизации состояния |

# Расчёты нагрузки (порядок величин)

Мы ориентируемся на **критичные уведомления**, которые должны доставляться с пропускной способностью ≥ 10 000 уведомлений/сек (NFR3). Рассмотрим базовые расчёты нагрузки для обоих вариантов:

**Входные данные:**

| Параметр | Значение |
|---|---|
| Средняя длина уведомления | 1 КБ |
| Максимальная скорость генерации уведомлений | 10 000 уведомлений/сек |
| Среднее время доставки через канал | 100 мс (push), 500 мс (SMS), 200 мс (email) |
| Количество каналов | 3 (push/SMS/email) |
| Retry | 3 попытки |
| Failover | ≤ 5 секунд |

**Расчёт нагрузки на очередь и воркеры (Вариант 1):**

- При пиковых 10 000 уведомлений/сек, и средней задержке доставки 100 мс:
  - Среднее количество уведомлений в обработке = 10 000 × 0,1 = **1 000 уведомлений** одновременно.
- Для failover и retry:
  - С учётом 3 повторов и возможности переключения канала, нагрузка может увеличиваться до **≈3 000 сообщений** в очереди Dead Letter Queue.
- Каждый воркер при такой нагрузке обрабатывает 1–3 тыс. уведомлений в секунду, что позволяет горизонтально масштабировать систему.

**Расчёт нагрузки для микросервисной архитектуры (Вариант 2):**

- Routing Service обрабатывает 10 000 уведомлений/сек.
- Хранилище состояния (Redis/Cassandra) должно выдерживать запись статусов доставки ≥ 30 000 операций/сек (10 000 × 3 попытки).
- Канальные сервисы масштабируются горизонтально, обеспечивая пропускную способность ≥ 10 000 уведомлений/сек.

# Ключевые компромиссы и ограничения

| Компонент | Компромисс | Ограничение |
|---|---|---|
| Очередь с приоритетами | Простая маршрутизация, высокая надёжность | Более высокая сложность при массовых рассылках, требуются дополнительные воркеры |

| Компонент | Компромисс | Ограничение |
|---|---|---|
| Микросервисная маршрутизация | Гибкая маршрутизация, независимая масштабируемость | Зависимость от распределённого хранилища состояния, возможны проблемы согласованности |
| Retry/Dead Letter Queue | Гарантированная доставка | Увеличение задержки при пиковых нагрузках |
| Failover | Отказоустойчивость | Дополнительная логика, возможны дублирования, если не синхронизировано состояние |

# Выводы и обоснование выбора

- **Рекомендованный вариант: Вариант 1 – Центральная очередь с приоритетами и failover**
- **Обоснование:**
  - Гарантированная доставка при отказах внешних провайдеров.
  - Приоритизация критичных уведомлений встроена на уровне очереди.
  - Простая интеграция и наблюдаемость (мониторинг, метрики, логирование).
  - Масштабируемость достигается горизонтально через количество воркеров, без сложной синхронизации состояния между микросервисами.
- **Основные компромиссы:**
  - Чуть более высокая сложность архитектуры и управление очередями.
  - Возможны задержки при пиковых массовых рассылках, которые компенсируются горизонтальным масштабированием.